

scores data to create one list of scores and another of names; both lists are returned.

Finally, the `getHighestAndLowest()` method ⑩ is called by the High Scores scene to get the highest and lowest scores in the table. It uses these results to determine if a user's score qualifies the user to enter their name and score into the high scores table.

Extensions to the Game

The overall architecture is modular, allowing for ease of modification. Each scene encapsulates its own data and methods, while communication and navigation are handled by the scene manager. Extensions can be handled in one scene without affecting anything in other scenes.

For example, rather than the game ending as soon as the Player icon hits a Baddie, you might want the user to start with some number of lives; when the Player icon hits a Baddie, the number of lives is decremented by one, and the game is over when the player runs out of lives. This kind of change would be relatively easy to implement and would affect only the Play scene.

As another idea, the user might start with a small number of bombs that they can detonate when they're in a bind, eliminating all Baddies within a given radius around the Player icon. The count of bombs would be decremented each time one is used, until it reaches zero. This change would affect only the code of the Play scene and the Baddie manager.

Or maybe you want to keep track of more high scores—say, 20 instead of 10. A change like that can be made in the High Scores scene without affecting the Play or Splash scenes.

Summary

This chapter demonstrated how to create and use Yes/No and Answer dialogs—both text and customizable versions. We then focused on building a full object-oriented game program, Dodger.

We used the `pygwidgets` module for all the buttons, text displays, and input text fields. We used the `pyghelpers` module for all the dialogs. The `SceneMgr`